

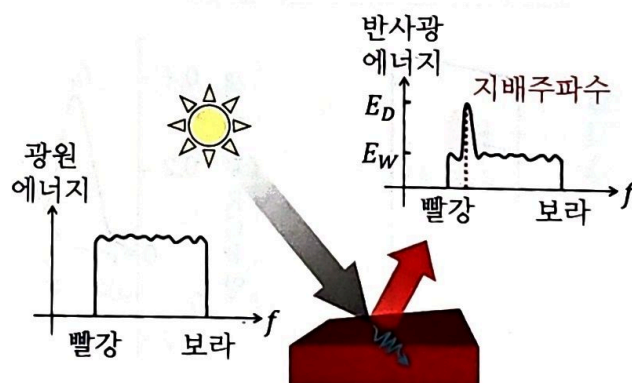
4.1 색 모델

4.1.1. 색의 성질

컴퓨터 그래픽스에서 색은 물체를 표현하는 데 중요한 요소이다. 색은 태양이나 전등과 같은 광원을 통해 전달된 빛이 물체의 표면에서 반사되거나 물체 스스로 내는 빛을 눈을 통해 감지한 결과이다. 사람의 눈으로 볼 수 있는 빛, 즉 가시광선은 일정한 주파수 범위의 전자기파로 약 4.0×10^{14} Hz부터 약 7.9×10^{14} Hz 범위에 해당되는 영역의 빛이다. 낮은 주파수 영역에서는 빨강, 높은 주파수 영역에서는 보라색으로 느낀다.

태양과 같은 광원은 가시영역 내의 모든 주파수를 복사함으로써 흰색의 빛을 만들어 낸다. 흰빛이 불투명한 물체에 입사되면 일부 주파수는 반사되고 나머지는 흡수된다. 이때, 사람이 그 물체를 보게 되는 색은 반사되는 빛의 색 조합이다.

색을 나타내는 세 가지 요소는 색조, 채도, 명도이다. [그림 4-1]은 백색 광원이 붉은 물체에 반사되었을 때 반사광의 에너지를 보여 주는 그림으로 전체 범위에 E_W 만큼의 에너지가 분포하며, 빨강 부근의 특정 주파수에서 큰 에너지(E_D)를 반사하는 것을 볼 수 있다. 이때, 이 강한 에너지가 반사광의 색을 결정하는데, 이에 해당되는 주파수를 지배주파수(dominant frequency, 또는 우세주파수)라고 하며, 지배주파수에 해당되는 색 성분을 색조(hue)라고 한다.



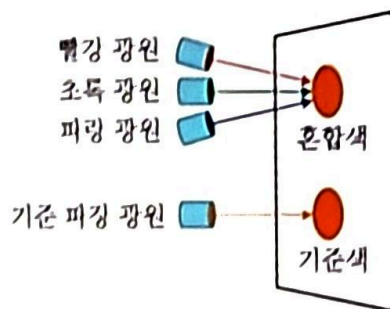
[그림 4-1] 빛의 에너지 분포와 지배주파수

[그림 4-1]에서 전체 가시주파수 영역에 분포하는 크기 E_W 의 에너지는 반사광의 흰색 성분에 해당된다. 이 성분이 강해지면 색은 점점 얼어져서 파스텔 톤을 내게 된다. 따라서 이 성분이 약할수록 선명한 색을 내게 된다. 채도(saturation)는 색이 순수한 스펙트럼색에 얼마나 가까운가를 나타내는 성분으로 $E_D - E_W$ 가 클수록 채도가 높다.

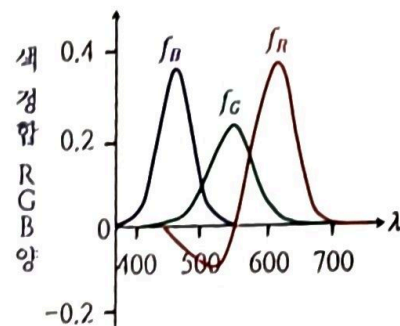
명도(brightness, luminance)는 빛의 전체적인 에너지 크기에 해당되는 요소이다. 이러한 색을 사람의 시각이 감지하는 것은 눈의 망막에 있는 시각세포가 담당한다. 특히 색에 반응하는 원추세포는 빛의 파장에 따라 흡수도가 다른 세 종류의 광색소가 있으며, 각각 440nm, 545nm, 580nm 근방에서 흡수율이 가장 높은 것으로 알려져 있다. 이때, 사람이 느끼는 색은 이 세 가지 광색소에 흡수되는 빛의 양 r, g, b 에 따라 결정되는데, 이를 삼원자극이론(tri-stimulus theory)이라고 한다. 이는 감지되는 빛의 양 r, g, b 가 같다면 색을 구성하는 주파수 성분이 다르더라도 같은 색으로 인지될 수 있다는 것을 의미한다.

컴퓨터나 TV 등에서 컬러 화면을 출력할 때는 이러한 원리에 따라 빨강(R), 초록(G), 파랑(B)의 3가지 파장의 빛을 혼합하여 색을 표현한다. 이때, 이같이 선택된 원색을 혼합해 만들어 낼 수 있는 모든 색의 집합을 색역(color gamut, 색 영역)이라 한다.

가시주파수 내의 순수한 스펙트럼색을 내기 위해 필요한 RGB 색 성분의 강도를 구하기 위해 [그림 4-2]의 (가)와 같은 색 정합(color matching) 실험을 할 수 있다. 우선 원하는 기준 파장의 레이저를 이용하여 기준색을 쏜다. 동시에 R, G, B 광원의 강도를 각각 조정하여 빛을 혼합한다. 이때, 혼합된 색이 기준색과 동일하게 보이는 R, G, B 강도가 [그림 4-2]의 (나)이다.



(가) 색 정합 개념도

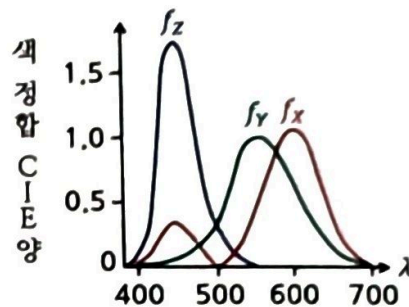


(나) 색 정합 함수

[그림 4-2] 색 정합

이거서 R 성분의 강도 f_R 가 음이 되는 영역이 있는데, 이는 해당 영역의 색을 R, G, B 의 혼합으로는 만들어 낼 수 없고, 거꾸로 R 성분의 음의 세기에 해당되는 빛을 기준 파장 광원에 더해 줌으로써 $R=0$ 인 혼합색과 같은 색을 만들 수 있음을 나타낸다. 이는 우리가 사용하는 RGB 모니터가 표현할 수 있는 색역이 실세계에 존재하는 모든 색을 포함하지 못한다는 것을 의미한다.

CIE(국제조명위원회)는 1931년 R, G, B 대신 X, Y, Z 라는 가상의 표준원색을 인간의 시각 특성에 맞추어 수학적으로 정의하였다. 비록 현실적으로 이러한 색의 자연광원이 존재하지 않는 가상의 색이지만, X, Y, Z 는 각각 빨강, 초록, 파랑에 가까운 색이며, 이를 혼합하여 평균적인 인간의 눈에 보이는 모든 색을 표현할 수 있다. [그림 4-3]은 CIE XYZ 색 공간의 가상원색을 보여 준다.



[그림 4-3] CIE 가상원색

CIE XYZ 색 공간에서 Y 성분은 밝기의 값이 되도록 설계되어 있다. X, Y, Z 를 정규화하여 x, y, z 로 색 성분을 표현할 수 있으며, x, y, z 는 각각 다음과 같이 계산할 수 있다.

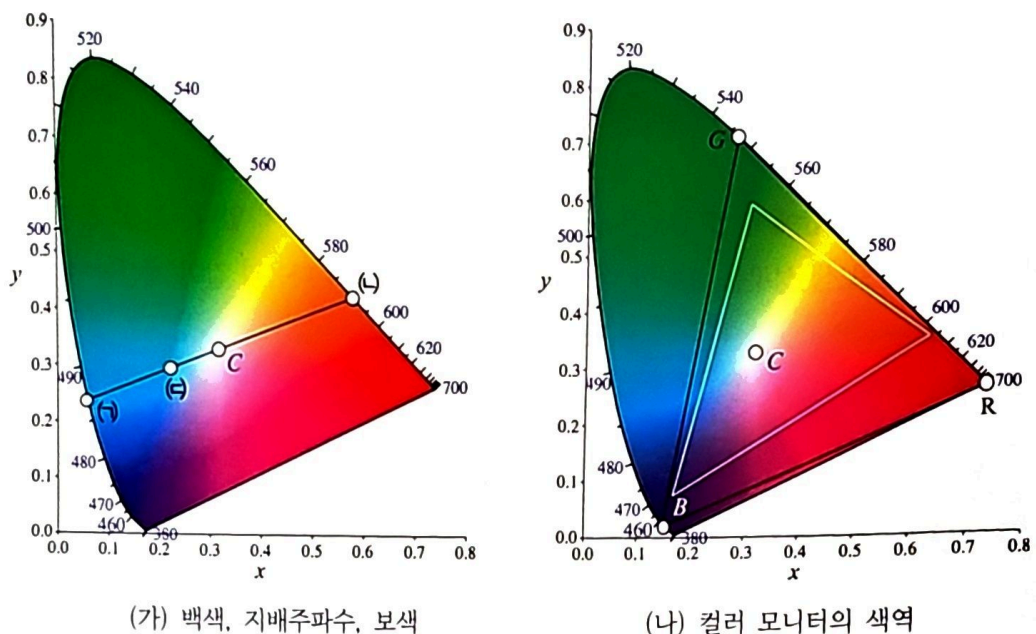
$$\begin{aligned} x &= \frac{X}{X+Y+Z} \\ y &= \frac{Y}{X+Y+Z} \\ z &= \frac{Z}{X+Y+Z} = 1 - x - y \end{aligned} \quad (4-1)$$

여기서 $x+y+z=1$ 이므로 z 는 x, y 로부터 계산할 수 있다. CIE XYZ 색 공간에서 Y 성분은 밝기의 값이 되도록 설계되어 있으므로 x, y 와 Y 값만 있으면 X, Y, Z 값을

모두 구할 수 있으며, 밝기값인 Y 를 제외한 나머지 성분인 (x, y) 만으로 밝기 성분을 제외한 나머지 색 정보를 표현할 수 있다. [그림 4-4]의 (가)는 임의의 색상의 (x, y) 좌표를 표현한 그림이다. 그림에서 가장자리에 위치한 색은 모두 표시된 파장에 해당되는 스펙트럼색에 해당되는 순수한 색상이며, 내부의 색상은 이러한 색들의 조합으로 표현된다.

[그림 4-4] (가)의 CIE 색도표에서 중앙의 C 지점은 태양빛에 해당되는 백색광을 나타낸다. C 지점으로부터 가장자리 위치로 갈수록 채도가 높아지며, 가장자리 선상의 색은 그 지점과 C 사이를 잇는 선상의 색에 대한 지배주파수에 해당되는 색이다. 즉, (c) 지점 색의 지배주파수는 (γ) 지점의 색에 해당된다. 또한 어떠한 색 지점과의 평균 위치가 C 가 되는 점은 그 색의 보색이 된다. 즉, (γ) 은 (ι) 과 보색 관계에 있는 것이다.

그림에서 보는 것처럼 임의의 세 스펙트럼 원색을 조합하여 모든 가시광선의 색을 만들어 내는 방법은 없다. 실제로 이상적인 RGB 광원을 활용하더라도 [그림 4-4] (나)의 R, G, B 를 잇는 삼각형 내부의 색만 표현할 수 있을 뿐이다. 일반적인 RGB 컬러 모니터의 경우 이보다 좁은 범위의 색역을 갖는다.



[그림 4-4] CIE 색도표

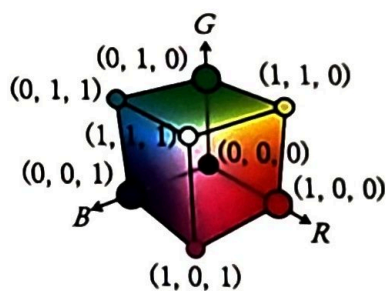
4.1.2. 여러 가지 색 모델

CIE XYZ 색 모델은 색 표현의 일반적인 틀을 제공하지만, 컴퓨터 그래픽스 시스템에서 제공할 수 있는 색역은 그 일부에 해당되며, 출력장치의 특성에 맞는 색 모델을 채택하여 사용하게 된다. 컴퓨터 그래픽스에서는 RGB 모델, CMY 모델, HSV 모델 등 다양한 색 모델이 사용된다.

(1) RGB 색 모델

RGB 색 모델은 R (빨강), G (초록), B (파랑)를 삼원색으로 하는 색 모델로서 삼원자 극이론에 근거한 색 모델이다. [그림 4-5]의 (가)는 R, G, B 의 세 축으로 표현된 RGB 색 공간이다. $(1, 0, 0)$ 은 빨강, $(0, 1, 0)$ 은 초록, $(0, 0, 1)$ 은 파랑을 나타내며, 세 성분의 값을 적절한 비율로 섞어 원하는 색을 만든다. 예를 들어, 노랑은 빨강과 초록을 더하여 만들며, 청록은 파랑과 초록을 더하여 만든다. 빨강과 초록, 파랑을 모두 동일한 비율로 섞으면 검정부터 하양에 이르는 회색조 색상이 나온다.

이처럼 RGB 색 모델은 (R, G, B) 삼원색이 더해져서 혼합색을 만들어 낸다. 그러므로 RGB 색 모델은 가산색 모델이라고 한다. 특히 출력장치가 스스로 빛을 냄으로써 색을 표현한다면 가산색 모델이 적합하다. 그러므로 RGB 색 모델은 CRT나 LCD 모니터 등과 같이 빛을 더하여 색을 표현하는 디스플레이 장치에 적용된다.



(가) RGB 색 공간



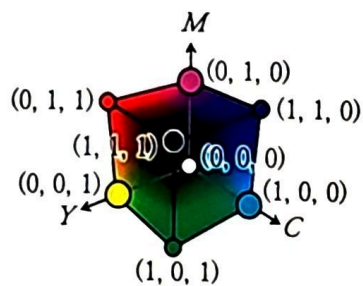
(나) 가산색의 혼합

[그림 4-5] RGB 색 모델

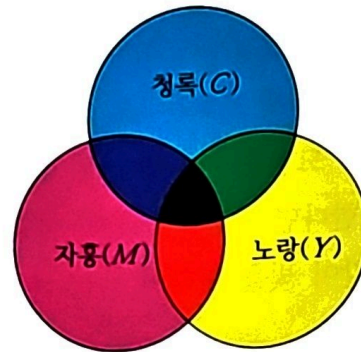
(2) CMY 색 모델

CMY 색 모델은 C (Cyan, 청록), M (Magenta, 자홍), Y (Yellow, 노랑)의 세 가지 색을 원색으로 하는 색 모델이다. RGB 색 모델이 가산색 모델인 반면, CMY 색 모델은 감산색 모델로 스스로 빛을 내는 출력장치가 아니라 외부로부터 가해진 빛을 반사하는 형태의 출력과 관련된 색 모델이다.

감산색 모델의 경우, 포함된 색 성분에 해당되는 빛을 흡수하는 특성이 있다. 즉, 백색광이 가해졌을 때 아무런 색이 가해지지 않으면 모든 빛을 반사하여 흰색으로 보이지만, 청록(C)색을 가하면 빨강 성분을 흡수하여 청록색으로 보인다. 마찬가지로 자홍(M)은 초록 성분을, 노랑(Y)은 파랑 성분을 흡수하는 원색이다. 청록(C)과 자홍(M)을 혼합하면 빨강과 초록 성분을 흡수하여 파랑으로 보이며, 모든 색을 혼합하면 빨강, 초록, 파랑 성분을 모두 흡수하여 검정이 된다.



(가) CMY 색 공간



(나) 감산색의 혼합

[그림 4-6] CMY 색 모델

CMY 색 모델은 컬러 프린터와 같은 출력장치에 사용된다. CMY 모델과 RGB 모델 사이의 변환은 식 (4-2)와 같다.

$$\begin{bmatrix} C \\ M \\ Y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} R \\ G \\ B \end{bmatrix} \quad \begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} C \\ M \\ Y \end{bmatrix} \quad (4-2)$$

그러나 이상적으로 색을 흡수하는 잉크는 없으며, 그 결과 의도하였던 색의 흡수는 이루어지지 않는다. 따라서 RGB 컬러 모니터로 표시된 그림을 프린터로 인쇄할 경우,

모니터의 화면과 동일한 출력물을 얻지 못한다.

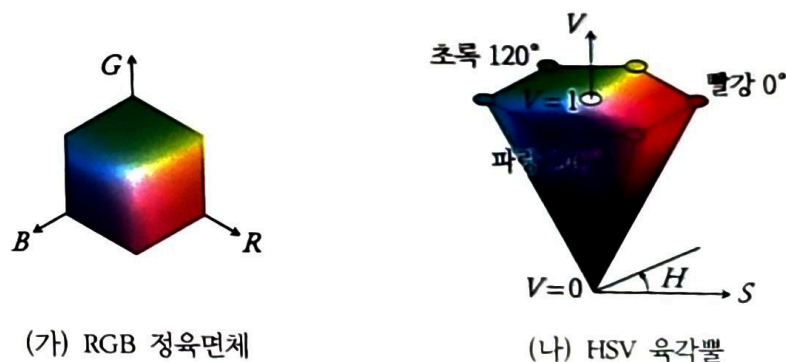
CMY 모델을 확장한 색 모델로 CMYK 색 모델이 있다. 이 모델에서는 CMY 색 모델에 회색의 농도를 결정하는 K 성분이 추가된다. 만일 잉크젯 프린터로 CMY 색 모델에서 $C=0.7$, $M=0.9$, $Y=1.0$ 으로 출력하고자 한다면, 이에 해당되는 농도로 C, M, Y 잉크를 뿌려 출력해야 한다. 그런데 이러한 출력은 농도 0.7의 검정 잉크와 함께 농도 0.2의 자홍, 농도 0.3의 노랑 잉크를 출력하는 것과 같다. C, M, Y를 농도 0.7로 뿌려 동일한 회색조를 얻는 것은 앞서 언급한 색 흡수의 한계로 인해 어렵지만, 이와 같이 검정 잉크를 사용하면 훨씬 용이해진다.

(3) HSV 색 모델

RGB나 CMY 색 모델이 출력장치를 고려한 삼원색을 사용하여 색을 표현하는 모델이라면, HSV 색 모델은 사람이 생각할 수 있는 직관적인 개념에 좀 더 근접한 모델이다. 앞서 색을 나타내는 세 가지 요소인 색조(Hue: H), 채도(Saturation: S), 명도(brightness 또는 Value: V)에 대해 언급한 바 있는데, HSV 모델은 이를 이용하여 색을 표현한다.

[그림 4-5] (가)의 RGB 정육면체 색 공간을 (1, 1, 1)에서 (0, 0, 0) 방향으로 바라보면 [그림 4-7]의 (가)와 같은 정육각형으로 보일 것이다. HSV 색 모델에서는 이 정육각형을 $V=1$ 인 면으로 하고, 아래로 내려갈수록 어두워지는 [그림 4-7]의 (나)와 같은 육각뿔로 색을 모델링한다.

$V=1$ 인 면을 기준으로 할 때, 중앙은 $S=0$ 으로 채도가 가장 낮은 지점인 백색에 해당된다. 채도의 축은 중앙에서 빨강 방향이며, 이 지점을 기준으로 시계 반대방향의



[그림 4-7] HSV 색 모델

각도가 색조 성분에 해당된다. RGB의 삼원색에 해당되는 빨강·초록·파랑은 각각 0° , 120° , 240° 에 위치한다. 육각뿔의 아래로 내려갈수록 어두운 색이 되며, 가장 아래의 정점인 $V=0$ 은 검정에 해당된다.

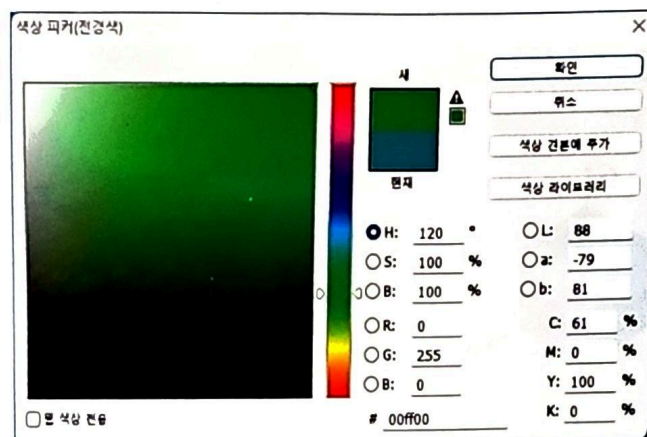
RGB 모델과 HSV 모델은 서로 간단하게 변환된다. 많은 그래픽스 패키지에서 RGB 모델과 HSV 모델을 자동변환하는 기능을 제공하여 두 모델 중 원하는 모델에 따라 색을 지정할 수 있도록 하고 있다.

(4) 그 밖의 색 모델

이외에도 다양한 색 모델이 정의되어 사용된다. 특히 비디오 신호와 관련하여 YIQ, YUV, $YCbCr$ 등의 색 모델이 사용된다. YIQ 색 모델은 우리나라와 북아메리카에서 아날로그 TV에 사용되는 NTSC 방식의 복합 비디오 신호를 만들기 위한 색 부호화 모델이며, YUV는 유럽의 PAL 방식에서 사용되는 색 모델이다. $YCbCr$ 은 디지털 TV의 색 모델로 활용되고 있다. Y 성분은 밝기에 해당되며, 나머지 요소는 색을 표현하기 위해 사용된다.

이러한 모델들은 그림을 저장하거나 전송할 때 제한된 용량에 맞게 색을 표현하는데 용이하다. 눈에 가장 민감한 밝기 성분(Y)에 가장 많은 정보량을 배분하고, 상대적으로 덜 민감한 색 성분에 대해서는 적은 정보량을 배분하여 색을 표현하는 것이다.

이와 같이 하면 더 적은 정보량으로 그림을 표현하면서도 시각적으로는 큰 차이를 느끼지 않게 할 수 있다. 또한 밝기 성분을 색 성분과 분리하여 표현함으로써 과거의



[그림 4-8] 여러 가지 색 모델을 통한 색상지정 기능(Adobe Photoshop CS6)

후백 시스템과의 호환을 유지할 수 있다.

4.2 그래픽스 기본 요소의 속성

색이나 두께 등 그래픽스 기본 요소의 특성을 설정할 수 있는 파라미터를 속성 파라미터(attribute parameter)라고 한다. 그래픽스 패키지에서 기본 요소에 속성 옵션을 결합하는 한 가지 방법은 각 그래픽스 기본 요소 함수의 파라미터 목록에 적절한 속성값을 포함하도록 함수를 정의하는 것이다. 예를 들어, 선분 그리기 함수의 경우 선분의 색상, 굵기, 기타 특성을 지정하는 값이 함수의 파라미터를 통해 전달되도록 정의하는 것이다. 이 방법은 함수 호출 시 전달해야 할 파라미터가 많아지고, 동일한 속성값을 반복적으로 지정하는 경우에도 계속 속성값을 전달해야 한다.

다른 방법으로는 현재 속성값에 대한 시스템 목록을 가지고 있는 것이다. 이 경우 속성 목록의 현재 값을 설정하기 위한 별도의 함수가 그래픽스 패키지에 포함된다. 그래픽스 기본 요소를 디스플레이하려면 시스템은 관련된 속성을 검사하고, 현재의 속성 설정을 사용하여 그 기본 요소에 대한 디스플레이 루틴을 호출한다.

일부 그래픽스 패키지들은 전자와 같은 방식으로 그래픽스 기본 요소의 속성을 지정한다. 반면에, OpenGL을 포함한 많은 그래픽스 라이브러리는 후자와 같이 시스템 속성 리스트를 갱신하는 별도의 함수를 사용하여 속성을 지정한다.

현재의 속성값과 기타 파라미터의 리스트를 유지하고 있는 그래픽스 시스템을 상태 시스템(state system) 또는 상태 기계(state machine)라고 한다. 출력 기본 요소의 속성이나 현재의 프레임 버퍼 위치와 같은 기타 파라미터들은 상태 변수(state variables) 또는 상태 파라미터(state parameters)라고 한다. 우리가 1개 이상의 상태 파라미터에 값을 할당하면 시스템은 특정 상태가 된다. 또한 그 상태는 상태 파라미터를 변경할 때까지 효력을 유지한다.

4.2.1. 색과 그레이 스케일

색상을 표현하기 위한 여러 가지 색 좌표계가 제시되고 있지만, 컬러 디스플레이 장치에 가장 자연스러운 색 모델은 [그림 4-5]의 RGB 색 모델이다. 레스터 그래픽스 시스템에서는 주사변환을 통해 한 화면을 픽셀들의 집합으로 표현한다. 프레임 버퍼에는 각 픽셀에 대한 색 정보가 저장된다.

한 시스템에서 선택할 수 있는 색의 수는 프레임 버퍼 내에서 한 픽셀당 할당되는 저장공간의 크기에 따라 결정된다. 색 정보를 프레임 버퍼에 저장하는 방법은 두 가지로 나눌 수 있다. 하나는 RGB 색 부호를 프레임 버퍼에 직접 저장하는 것이고, 또 하나는 색 부호를 별도의 테이블에 넣어 둔 후 프레임 버퍼에는 색 테이블 항목을 참조하는 인덱스값을 저장하는 것이다.

직접저장 방식을 사용할 경우 디스플레이될 출력 원시요소의 각 픽셀에 해당되는 프레임 버퍼 위치에 그 픽셀의 색 정보가 저장된다. 이때, 하나의 픽셀에 얼마나 많은 색을 표현할 수 있게 할 것인가에 따라 프레임 버퍼에 요구되는 저장공간이 결정된다. 픽셀당 3비트로 표현할 경우 <표 4-1>과 같이 8가지의 색을 출력할 수 있다. 더 많은 색을 표현하고자 한다면 더 많은 비트가 필요하다. 오늘날 흔히 ‘트루 컬러’, ‘풀 컬러’ 등으로 부르는 그래픽스 장치에서는 R, G, B 각 성분당 8비트씩 총 24비트가 사용되며, 색 혼합을 위한 알파 계수를 추가하여 32비트가 사용되기도 한다. 이 경우 화면 해상도가 1024×1024 이면 한 장의 그림을 저장하기 위해 3MB 또는 4MB의 프레임 버퍼가 필요하다. 좀 더 높은 정밀도로 색을 표현하는 시스템의 경우에는 R, G, B의 각

<표 4-1> 픽셀당 3비트인 프레임 버퍼의 8가지 색 부호

색 부호	프레임 버퍼에 저장된 색값			디스플레이된 색
	빨강	초록	파랑	
0	0	0	0	검정
1	0	0	1	파랑
2	0	1	0	초록
3	0	1	1	청록
4	1	0	0	빨강
5	1	0	1	자홍
6	1	1	0	노랑
7	1	1	1	흰색

성분당 12비트가 사용되기도 한다.

색 테이블을 이용하여 색을 표현하는 방식에서는 색 테이블 안에 그림에서 사용할 색의 R , G , B 값을 저장해 놓는다. 프레임 버퍼에는 픽셀의 색을 직접 저장하는 대신 색 테이블의 위치를 가리키는 인덱스가 저장된다. 흔히 256개의 항목으로 구성된 색 테이블을 사용하며, 이 경우 그림에서 표현할 수 있는 색의 종류는 256개로 제한된다. 이와 같이 하면 화면 해상도가 1024×1024 인 경우 한 장의 그림을 저장하기 위한 프레임 버퍼의 크기는 1MB이면 된다.

이러한 방식은 커다란 프레임 버퍼를 사용하지 않고 방대한 색 성능을 제공하기 위한 대체 수단이며, 한때는 이 방법이 중요하게 고려되었다. 그러나 요즘은 하드웨어 비용이 급격히 하락하여 저가의 개인용 컴퓨터 시스템조차도 방대한 색 성능을 가지고 있는 것이 일반적이다. 따라서 대부분의 예에서는 단순히 RGB 색 부호가 프레임 버퍼에 직접 저장된다고 가정할 것이다.

색 기능은 이제 컴퓨터 그래픽스 시스템에서 일반적이므로 응용 프로그램에서 그레이 음영 또는 그레이 스케일(gray scale)을 지정하기 위해 RGB 색 함수를 사용한다. RGB 색 설정을 동일한 적색, 녹색, 청색 양을 지정하면 그 결과는 그레이 음영이 된다. 색 성분이 0에 가까운 값을 가질수록 어두운 회색이 되며, 1.0 부근의 값은 밝은 회색을 만들어 낸다.

4.2.2. OpenGL 색 함수

(1) 꼭짓점의 색 지정

OpenGL의 색 표현은 기본적으로 $[0.0, 1.0]$ 범위의 float형으로 RGB 및 알파값을 지정한다. 즉, 가장 밝은 빨강은 $(1.0f, 0.0f, 0.0f, 1.0f)$ 이며, 중간 밝기의 청록은 $(0.0f, 0.5f, 0.5f, 1.0f)$ 이다.

만일 정수형으로 색을 표현할 경우 0이 가장 어두운 값, 그 자료형으로 표현할 수 있는 가장 큰 값이 가장 밝은 값을 나타낸다. 즉, GLubyte로 색을 표현한다면 앞의 두 색은 각각 $(255, 0, 0, 255)$ 와 $(0, 128, 128, 255)$ 로 표현할 수 있으며, GLshort로 표현한다면 $(32767, 0, 0, 32767)$ 과 $(0, 16384, 16384, 32767)$ 이다. 이렇게 표현된 색은 셰이더로 전달할 때 $[0.0, 1.0]$ 범위로 정규화한다.

OpenGL에서 색을 지정하는 것은 여러 가지 방법을 이용할 수 있다. 한 가지 방법은

OpenGL의 명령 없이 셰이더에서 직접 색을 지정하는 것이다. 제2장의 [소스코드 2-1]에서는 프래그먼트 셰이더에서 픽셀의 색을 빨강으로 직접 지정하였다(28행).

다른 방법은 꼭짓점 데이터를 셰이더에 전달할 때 좌표와 함께 색을 전달하는 것이다. 즉, 꼭짓점 비피에 꼭짓점 좌표와 더불어 색을 저장하고, 이를 셰이더에 전달하는 것이다. [소스코드 4-1]은 꼭짓점에 색을 지정하여 삼각형을 출력하는 예를 보여 주는 프로그램이다.

[소스코드 4-1] VertexColor.c

```
1  #include <stdio.h>
2
3  #include <gl/glew.h>
4  #include <gl/freeglut.h>
5
6  typedef struct {          // 3차원 좌표를 표현하기 위한 구조체
7      float x, y, z;
8  } Vec3f;
9
10 typedef struct {          // RGBA 색을 표현하기 위한 구조체
11     float r, g, b, a;
12 } VecColor4f;
13
14 typedef struct {
15     Vec3f      mCoord;
16     VecColor4f mColor;
17 } Vertex;
18
19 enum { TRIANGLE, N_VBOs };
20 GLuint VBO[N_VBOs];          // 꼭짓점 버퍼 객체
21
22 static const char* pVS =      // 꼭짓점 셰이더 소스
23     "#version 330\n"
24     "layout (location = 0) in vec3 Position;\n"
25     "layout (location = 1) in vec4 VColor;\n"
```



```

26     "out vec4 Color;                                \n"
27     "                                                \n"
28     "void main( )                                    \n"
29     "{                                                \n"
30     "    gl_Position = vec4(Position*0.1, 1.0);      \n"
31     "    Color = VColor;                              \n"
32     "};";
33
34     static const char* pFS = // 프래그먼트 셰이더 소스
35     "#version 330                                \n"
36     "in vec4 Color;                                \n"
37     "out vec4 FragColor;                            \n"
38     "                                                \n"
39     "void main( )                                    \n"
40     "{                                                \n"
41     "    FragColor = Color;                          \n"
42     "};";
43
44     void AddShader(GLuint shaderProg,
45                     const char* pShaderSrc, GLint ShaderType)
46     { // 셰이더 생성
47     ...     ※ [소스코드 2-1]과 동일함
48     }
49
50     void SetUpShaders( )
51     { // 셰이더 프로그램 객체 생성
52     ...     ※ [소스코드 2-1]과 동일함
53     }
54
55     void RenderCB( )
56     {
57         glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
58         glClear(GL_COLOR_BUFFER_BIT); // 백색으로 화면 지움
59
60         glEnableVertexAttribArray(0);

```

```

107     glEnableVertexAttribArray(1);
108     glBindBuffer(GL_ARRAY_BUFFER, VBO[TRIANGLE]);
109     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex), 0);
110     glVertexAttribPointer(1, 4, GL_FLOAT, GL_FALSE, sizeof(Vertex),
                            (const GLvoid*)sizeof(Vec3f));
111     glDrawArrays(GL_TRIANGLES, 0, 3);
112     glDisableVertexAttribArray(0);
113     glDisableVertexAttribArray(1);
114     glFinish();
115 }
116
117 void InitVBOs()
118 {
119     Vertex Vertices[3] = {      // 삼각형의 꼭짓점 좌표 및 색
120         { {-5.0f, -5.0f, 0.0f}, {1.0f, 0.0f, 0.0f, 1.0f} },
121         { {5.0f, -5.0f, 0.0f}, {0.0f, 1.0f, 0.0f, 1.0f} },
122         { {0.0f, 5.0f, 0.0f}, {0.0f, 0.0f, 1.0f, 1.0f} } };
123     // 꼭짓점 버퍼를 생성하여 삼각형의 꼭짓점 정보 전달
124     glGenBuffers(N_VBOs, VBO);
125     glBindBuffer(GL_ARRAY_BUFFER, VBO[TRIANGLE]);
126     glBufferData(GL_ARRAY_BUFFER, sizeof(Vertices), Vertices,
                  GL_STATIC_DRAW);
127 }
128
129 int main(int argc, char** argv)
130 {
131     glutInit(&argc, argv);
132     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
133     glutInitWindowPosition(50, 100);
134     glutInitWindowSize(640, 480);
135     glutCreateWindow("Vertex Color");
136     ...    ※ 이하 [소스코드 2-1]과 동일함
153 }

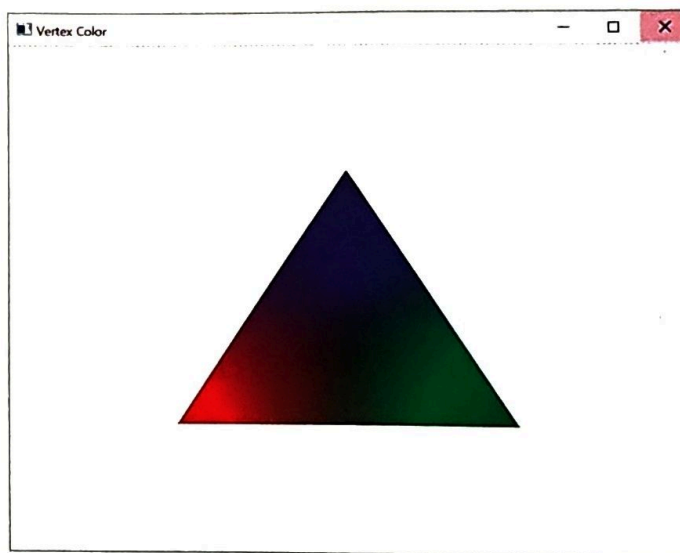
```

3차원 좌표를 표현하기 위한 구조체(6~8행)와 RGBA 색을 표현하기 위한 구조체(10~12행)를 선언한 후, 이들을 묶어 꼭짓점 정보를 나타내는 **Vertex**라는 구조체를 선언하였다. 이에 따라 함수 `InitVB0s`의 119~122행에서 3개의 꼭짓점 좌표와 색을 저장하는 **Vertex** 배열 데이터를 만들어 126행에서 꼭짓점 속성(`GL_ARRAY_BUFFER`)으로 전달하였다. 세 꼭짓점의 좌표와 색을 각각 $(-5.0, -5.0, 0.0)$ 에 빨강, $(5.0, -5.0, 0.0)$ 에 초록, $(0.0, 5.0, 0.0)$ 에 파랑으로 지정하였다.

이 꼭짓점 데이터는 꼭짓점 셰이더로 전달된다. 꼭짓점 셰이더는 전달되는 속성의 위치를 나타내는 인덱스로 좌표와 색을 구분하여 전달받는다. 24~25행에서 좌표를 입력하는 전역변수 `Position`은 0번 인덱스(`layout (location = 0)`)로, 색을 입력하는 전역변수 `VColor`는 1번 인덱스(`layout (location = 1)`)로 지정한 것을 볼 수 있다.

이에 맞추어 `RenderCB` 함수(101행)에서는 `glEnableVertexAttribArray` 함수로 0번과 1번 인덱스를 활성화한 후(106, 107행), 109행에서 `glVertexAttribPointer` 함수로 0번 인덱스에 꼭짓점 데이터의 선두를 기준으로 좌표가 저장되는 곳의 변위를(구조체 `Vertex`에서 좌표가 먼저 나오므로 0으로 지정하였음), 110행에서 1번 인덱스에 색이 저장되기 시작하는 곳의 변위를(좌표 다음이므로 좌표를 저장하는 데이터의 크기만큼 이동해야 하므로 `sizeof(Vec3f)`를 지정하였음. 단, `glVertexAttribPointer` 함수에서 변위를 `const GLvoid*`로 지정하도록 되어 있어 형변환을 지시하였음) 설정하였다.

꼭짓점 셰이더는 좌표를 입력받은 `Position`에 정해진 변환을 하여 내장변수 `gl_Position`



[그림 4-9] [소스코드 4-1]의 실행 화면

에 전달되도록 하였으며, 꼭짓점의 색을 입력받은 VColor의 값을 출력을 위한 전역변수로 선언한 Color(26행)를 통해 파이프라인의 다음 단계로 전달한다. 이러한 방식으로 꼭짓점 셰이더가 출력한 꼭짓점의 좌표와 색 데이터는 이후 단계의 처리를 거쳐 프래그먼트 셰이더에 입력으로 전달되며(36행), 셰이더 프로그램의 명령에 따라 이 색을 처리하여 FragColor라는 전역변수를 통해 출력된다.

[그림 4-9]는 [소스코드 4-1]의 실행결과이다. 삼각형 내부의 색은 꼭짓점 색을 보간하여 서서히 변화하는 색으로 출력한 것을 볼 수 있다.

(2) OpenGL 색 혼합

그림을 합성하거나 반투명한 물체를 표현하는 경우 물체의 색을 배경색과 혼합하는 기능을 사용한다. 그래픽스 패키지에서는 색 혼합을 하기 위해 사용할 수 있는 색 혼합 함수(color-blending functions)를 제공한다.

OpenGL에서도 현재 프레임 버퍼에 저장되어 있는 색에 새로운 물체의 색을 혼합하는 기능을 제공하는데, 현재의 프레임 버퍼 색을 목적지색(destination color)이라 하고, 두 번째 물체의 색을 소스색(source color)이라고 한다.

색 혼합 기능은 RGB나 RGBA 모드에서만 사용할 수 있는데, GL_BLEND 심벌 상수로 색 혼합 기능을 활성화하거나 해제할 수 있다.

```
glEnable(GL_BLEND);      // 색 혼합 기능 활성화
glDisable(GL_BLEND);     // 색 혼합 기능 해제
```

만일 색 혼합이 활성화되어 있지 않다면 단순히 물체 위치의 프레임 버퍼 내용이 그 물체의 색으로 대체된다.

색 혼합을 하기 위해서는 목적지색과 소스색에 대한 혼합인수 집합을 지정한다. RGBA 소스색 성분은 (R_s, G_s, B_s, A_s) , 목적지색 성분은 (R_d, G_d, B_d, A_d) , 소스 혼합인수는 (S_r, S_g, S_b, S_a) , 그리고 목적지 혼합인수는 (D_r, D_g, D_b, D_a) 라 할 때, 프레임 버퍼에 적재되는 새로운 혼합색 (R_m, G_m, B_m, A_m) 은 다음과 같이 계산된다.

$$\begin{aligned} & (R_m, G_m, B_m, A_m) \\ &= (S_r R_s + D_r R_d, S_g G_s + D_g G_d, S_b B_s + D_b B_d, S_a A_s + D_a A_d) \end{aligned} \quad (4-3)$$

혼합인수는 OpenGL에서 정한 상수로 표현된 값을 선택할 수 있다. `glBlendFunc` 함수는 이러한 용도로 사용하되, 사용할 수 있는 대표적인 인수를 지정하는 대표적인 상수는 다음과 같다.

```
void glBlendFunc(GLenum sFactor, GLenum dFactor);
```

- `sFactor`, `dFactor`: 소스 및 목적지 혼합인수로 아래의 값을 사용함
 - `GL_ZERO`: (0.0, 0.0, 0.0, 0.0), `dFactor`의 디폴트값임
 - `GL_ONE`: (1.0, 1.0, 1.0, 1.0), `sFactor`의 디폴트값임
 - `GL_SRC_ALPHA`: 소스 알파값으로 모든 인수값 지정
 - `GL_DST_ALPHA`: 목적지 알파값으로 모든 인수값 지정
 - `GL_ONE_MINUS_SRC_ALPHA`: (1-소스 알파값)으로 모든 인수값 지정
 - `GL_ONE_MINUS_DST_ALPHA`: (1-목적지 알파값)으로 모든 인수값 지정

(3) 그 밖의 OpenGL 색 함수

다음 함수는 색 버퍼의 내용을 지우는 값을 지정한다.

```
void glClearColor(GLclampf r, GLclampf g, GLclampf b, GLclampf a);
```

- `r`, `g`, `b`, `a`: 0.0부터 1.0 범위의 실수값으로 지정된 적, 녹, 청, 알파값

이후 지정된 색으로 프레임 버퍼를 지우는 것은 다음 함수를 사용한다.

```
void glClear(GLbitfield buf);
```

- `buf`의 값은 `GL_COLOR_BUFFER_BIT`, `GL_DEPTH_BUFFER_BIT`, `GL_ACCUM_BUFFER_BIT`, `GL_STENCIL_BUFFER_BIT`를 비트 단위 OR 연산(`|`)을 통해 지정

다음 명령은 색 버퍼의 내용을 지우는 값으로 흰색을 지정한 후 색 버퍼를 지운다.

```
glClearColor(1.0f, 1.0f, 1.0f, 1.0f);  
glClear(GL_COLOR_BUFFER_BIT);
```

4.2.3. 점 및 선 속성

(1) 점 속성

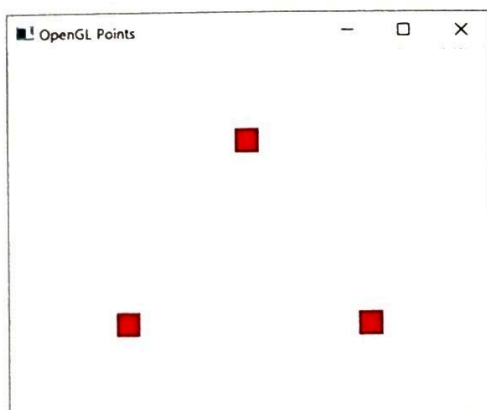
점에 대해서는 색과 크기 두 가지 속성을 설정할 수 있다. 색 성분은 RGBA 값으로 설정한다. 점의 크기는 `glPointSize` 함수를 이용하여 설정할 수 있으며, 표준 크기는 1이다. 지정할 수 있는 점의 최대 크기는 OpenGL의 구현에 따라 차이가 있으며, 그 크기보다 크게 지정된 점은 최대 크기로 출력된다. 커다란 점은 픽셀들의 정사각형 블록으로 디스플레이된다.

```
void glPointSize(GLfloat size);
```

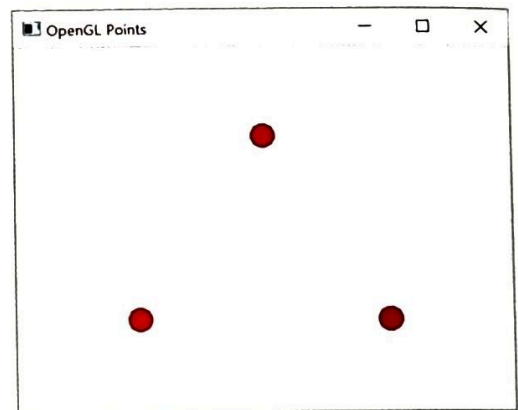
커다란 점이 디스플레이되는 모양은 점의 안티에일리어싱 활성화 여부에 따라 다르다. 다음과 같이 `glEnable` 함수로 점의 안티에일리어싱을 활성화하면 점 크기의 지름에 해당되는 원이 출력되고, 그렇지 않으면 픽셀들의 정사각형 블록으로 디스플레이된다.

```
glEnable(GL_POINT_SMOOTH); // 점의 안티에일리어싱 활성화
```

[그림 4-10]의 (가)는 점의 크기를 20.0으로 지정하여 그린 점이고, (나)는 동일한



(가) 크기를 20.0으로 지정한 점



(나) `GL_POINT_SMOOTH`를 적용한 점

[그림 4-10] 점 크기 속성의 적용

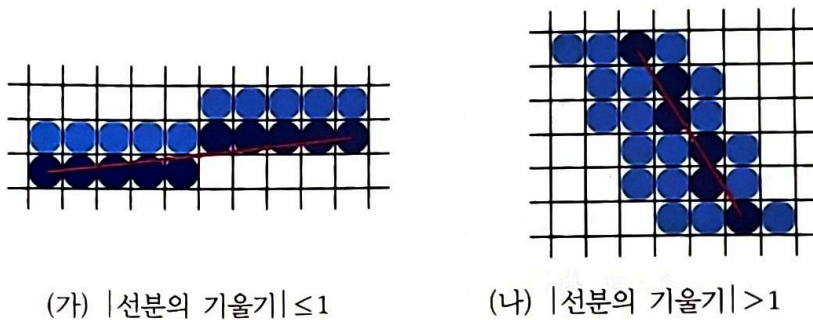
크기로 지정된 점을 GL_POINT_SMOOTH를 활성화하여 그린 것이다.

(2) 선 속성

선분은 색, 폭, 스타일이라는 세 가지 기본 속성을 설정할 수 있다.

① 선 폭

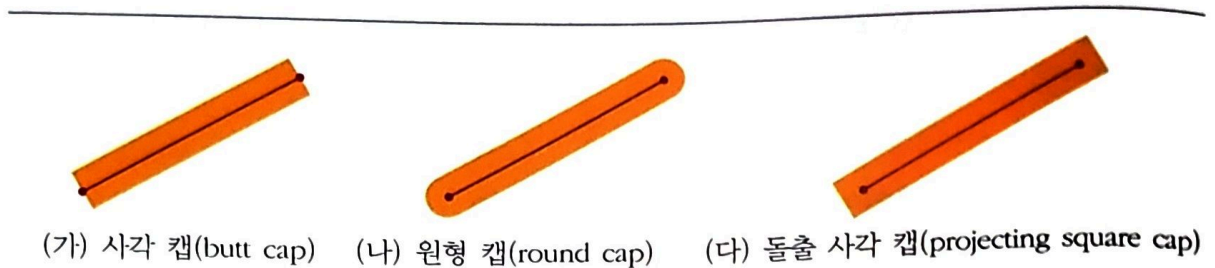
표준 선 폭은 선분의 주사변환에 의해 결정되는 각 샘플 위치에 하나씩의 픽셀로 그려진다. 이보다 굵은 선분의 경우에는 기울기의 크기가 1보다 작을 때는 직선 경로의 상·하에, 1보다 클 때는 직선 경로의 좌·우에 픽셀을 추가하여 그림으로써 표준 직선의 정수배 굵기의 선분을 그릴 수 있다. [그림 4-11]의 (가)는 기울기가 1보다 작고 폭이 2인 선분을 그린 결과이고, (나)는 기울기가 1보다 크고 폭이 4인 선분을 그린 결과이다. 곡선의 경우에도 유사한 처리를 함으로써 지정된 굵기로 그릴 수 있다.



[그림 4-11] 선 폭 속성의 구현

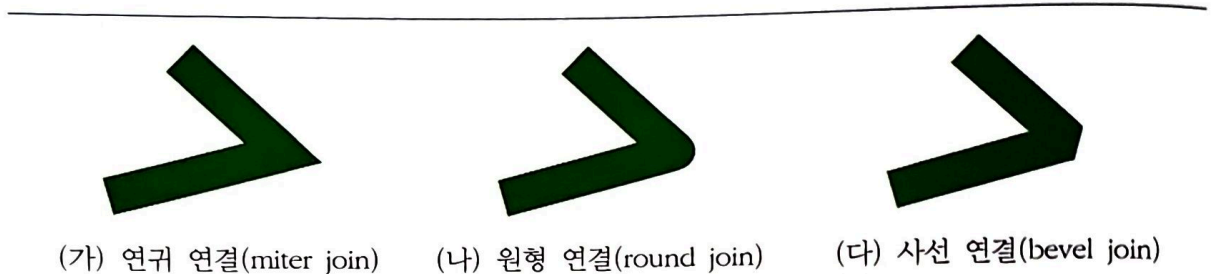
이러한 방법은 선의 폭을 빠르게 구현할 수는 있지만 정확하게 표현하지는 못한다. 특히 기울기의 크기가 1인 선분의 경우 다른 기울기의 선분에 비해 가늘게 보인다.

또한 [그림 4-11]의 (나)에서 볼 수 있는 것처럼 선의 끝 모양이 사선으로 잘리는데, 특히 폭이 굵은 선분일수록 두드러져 보인다. 그래픽스 패키지에 따라서는 [그림 4-12]와 같이 선분의 끝 모양(선분 캡)을 다양하게 표현한다.



[그림 4-12] 선분 캡의 종류

2개의 선분이 연결되는 부위에서도 유사한 문제가 발생한다. [그림 4-13]은 이러한 선분의 연결 부위를 다듬는 몇 가지 선분 연결 유형이다.



[그림 4-13] 선분 연결의 종류

선분의 폭을 좀 더 정확하게 구현하는 방법은 선분을 선 폭에 해당되는 직사각형으로 그린 후 내부를 선 색으로 채우는 것이다. 이와 같이 하면 선분의 기울기에 따라 선 폭이 다르게 보이는 현상을 방지할 수 있다.

OpenGL에서 선 폭을 지정하는 것은 다음의 `glLineWidth` 함수를 사용한다.

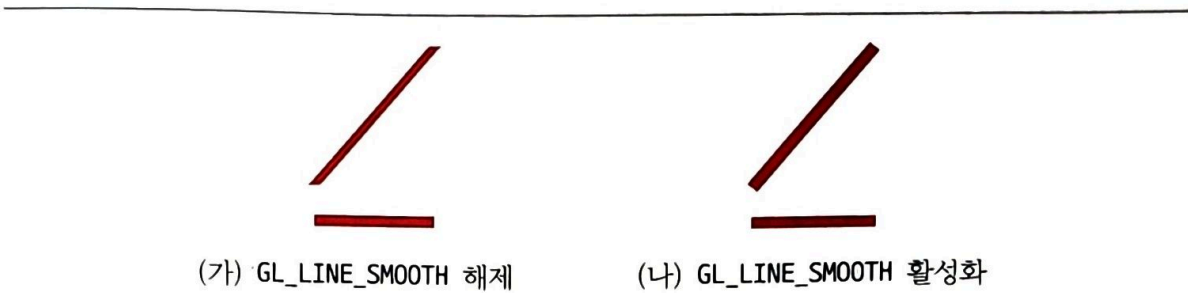
```
void glLineWidth(GLfloat width);
```

선을 그리면 기울기에 따라 `glLineWidth` 함수에서 지정한 폭만큼 선의 경로 좌·우 또는 상·하에 픽셀을 추가하여 선을 그린다. 그 결과, [그림 4-14]의 (가)와 같이 기울기에 따라 선 폭이 다르게 보일 수 있으며, 선분의 끝 모양은 선분의 기울기에 따라 수직 또는 수평으로 절단된 형태를 취한다.

`glEnable` 함수를 사용하여 선 그리기의 안티에일리어싱을 활성화할 수 있다.

```
glEnable(GL_LINE_SMOOTH);
```

이와 같이 하면 선분의 길이와 폭에 해당되는 직사각형 영역 내의 픽셀을 그린다. 그 결과 [그림 4-14]의 (나)와 같이 기울기와 관계없이 동일한 폭으로 선분을 그릴 수 있으며, 사각 캡 형태로 선분 끝 모양이 만들어진다.



[그림 4-14] OpenGL의 선 폭 지정효과(선 폭은 10.0으로 함)

② 선 스타일

선 스타일 속성으로는 실선, 점선, 파선 등이 있다. 이러한 속성은 선을 래스터화할 때 일정 구간의 픽셀을 건너뛰거나 그리는 것을 반복함으로써 구현할 수 있다.

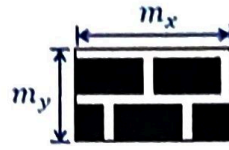
4.2.4. 영역 채우기 속성

다각형이나 임의의 경계에 의해 지정된 영역에 대한 속성을 지정할 수 있다. 영역의 속성은 단일색으로 지정될 수도 있지만, 여러 종류의 비트 패턴이나 텍스처 등이 될 수도 있다.

비트맵으로 영역을 채울 경우 패턴은 주로 직사각형 배열 형태의 마스크로 정의된다. 이 마스크는 대상 영역을 모두 채울 때까지 수평 및 수직 방향으로 반복적으로 복제하는 타일 붙이기(tiling) 방식으로 처리된다. [그림 4-15]는 $m_x \times m_y$ 크기의 벽돌 모양 비트맵 마스크의 예이다.

타일을 붙이기 시작하는 기준 위치를 정하는 방식은 두 가지로 나눌 수 있다.

첫째, 윈도우의 원점을 기준으로 타일 붙이기를 하는 것이다. 디스플레이 윈도우 원점이 (x_w, y_w) 인 경우 좌표 (x, y) 에 채울 색을 정하기 위한 비트맵 마스크의 좌표 (x_m, y_m)



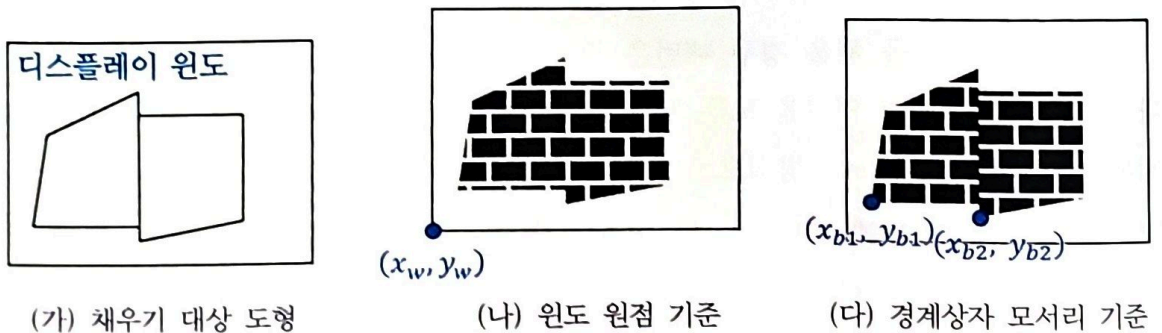
[그림 4-15] 비트맵 패턴 마스크

은 식 (4-4)와 같이 구할 수 있다.

$$\begin{aligned} x_m &= (x - x_w) \bmod m_x \\ y_m &= (y - y_w) \bmod m_y \end{aligned} \quad (4-4)$$

이 방법에서는 [그림 4-16]의 (가)와 같이 인접한 두 도형의 내부 패턴이 경계에서 일치하므로 마치 하나의 도형인 것처럼 보인다. 흔히 그래픽스 패키지에서 다각형을 정의할 때 블록다각형을 정의하는 것으로 가정하고, 오목다각형인 경우 블록다각형을 연결하여 정의하도록 하는데, 이러한 경우 원도 원점을 기준으로 내부를 채운다면 두 블록다각형의 경계가 나타나지 않아 하나의 도형인 것처럼 보일 수 있다는 장점이 있다. 그러나 이들이 별개의 다각형인 경우에도 하나처럼 보인다는 점과 다각형을 이동하는 애니메이션을 하는 경우 내부의 패턴이 멈추어 있는 상태에서 다각형이 움직이므로 다각형 내부의 패턴이 움직이는 것처럼 보일 수 있다는 단점이 있다.

둘째, 도형의 경계상자 모서리를 기준으로 타일 붙이기를 하는 것이다. 경계상자의 좌측 하단 모서리의 좌표가 (x_b, y_b) 인 경우 좌표 (x, y) 에 채울 색을 정하기 위한 비트맵 마스크의 좌표 (x_m, y_m) 은 식 (4-5)와 같이 구할 수 있다.



[그림 4-16] 비트맵 마스크의 타일 붙이기

$$\begin{aligned}x_m &= (x - x_b) \bmod m_x \\y_m &= (y - y_b) \bmod m_y\end{aligned}\tag{4-5}$$

[그림 4-16]의 (다)는 이 방법을 적용한 결과로, 두 다각형이 서로 다른 기준점에서부터 패턴을 채우므로 경계 부분의 패턴이 어긋나 서로 구분되어 보인다. 그 결과 첫째 방법과는 상반된 장점과 단점이 있다.

4.3 영역 채우기 알고리즘

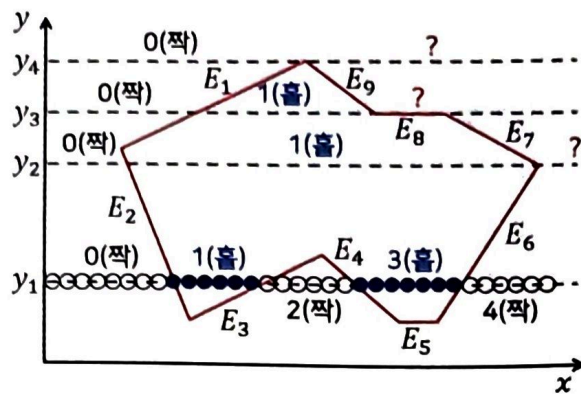
어떠한 영역을 지정된 색이나 패턴을 채우기 위해서는 영역의 경계가 어떻게 정의되는가에 따라 적절한 알고리즘을 선택해야 한다. 만일 어떠한 영역이 다각형이라면 주사선 단위로 다각형의 내부와 외부를 판별하는 처리를 통해 영역 채우기를 할 수 있다. 이러한 다각형이 볼록다각형이라면 처리는 더욱 단순화할 수 있다. 볼록다각형이 하나의 주사선과 만나는 교점은 2개만 존재하기 때문이다.

만일 영역의 경계가 불규칙한 곡선 형태인 경우는 영역 내부의 임의의 점에서 출발하여 인접한 내부 영역을 채우는 방식을 사용할 수 있다. 포토샵과 같은 페인트 패키지에서 어떠한 영역을 채우고자 할 때 이러한 방식을 흔히 사용한다.

4.3.1. 주사선 다각형 채우기

주사선 다각형 채우기 알고리즘은 각 주사선에 대해 주사선이 다각형의 변과 교차하는 점을 기준으로 다각형 내부 영역을 판별하여 픽셀을 그리는 방식이다. 이 알고리즘은 홀-짝 규칙을 사용하여 다각형의 내부와 외부를 판별하는 방법에 바탕을 두고 있다.

[그림 4-17]은 홀-짝 규칙으로 주사선의 다각형 내부 영역과 외부 영역을 판별하는 방법을 보여 준다. 주사선 y_1 을 좌에서 우로 주사하면 E_2 와 만나는 점에서 다각형 교차횟수가 1이 된다. 다음 E_3 와 만나는 점에서 교차횟수가 2, E_4 와 만나는 점에서 교차횟수가 3, E_6 와 만나는 점에서 교차횟수가 4이다. 이때, 교차횟수가 홀수인 영역을 다



[그림 4-17] 홀-짝 규칙(○: 외부 픽셀, ●: 내부 픽셀)

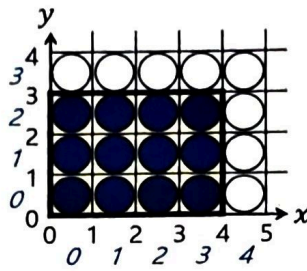
각형 내부로 판별하여 색이나 패턴을 채우면 된다.

문제는 주사선이 다각형의 꼭짓점을 만나는 경우이다. 주사선 y_2 는 E_6 및 E_7 과 만나므로 이 점을 지나면 교차 횟수가 3(홀)이 되어 다각형 내부로 잘못 판정하게 된다. 반면에, 주사선 y_4 의 경우 역시 꼭짓점을 만나는 경우이지만, 꼭짓점을 지나면 교차 횟수가 2(짝)가 되어 다각형 외부로 올바른 판정을 하게 된다. 이들의 차이는 y_2 의 경우 꼭짓점을 공유하는 두 변이 주사선의 위와 아래에 나뉘어 있고, y_4 의 경우 동일한 위치에 있다는 것이다. 전자의 경우에는 교차점을 지나면서 내부·외부의 상태가 바뀌게 되고, 후자의 경우에는 상태가 바뀌지 않아야 한다.

y_3 와 같이 x 축과 평행한 변을 만나는 경우에는 또 다른 문제가 제기된다. 변 위에 존재하는 점들을 내부로 처리할 것인가, 아니면 외부로 처리할 것인가의 문제인데, 이는 하나의 픽셀이 무한히 작은 점이 아니라 면적을 갖는 유한 크기 점이기 때문이다.

[그림 4-18]은 2개의 점 $(0, 0)$ 과 $(4, 3)$ 으로 정의된 직사각형이다. 이상적인 점의 좌표는 그림에서 격자선이 만나는 점이며, 이 점을 기준으로 $(0, 0)$ - $(4, 3)$ 의 직사각형을 그리면 그림의 붉은 선으로 표시된 영역이어야 하므로 그 면적은 4×3 이다. 그러나 실제로는 하나의 픽셀은 그림과 같이 면적을 갖는 점이며, 예를 들어, 격자의 0부터 1 사이의 범위가 픽셀 좌표 0이 되는 형태가 된다.

따라서 $0 \leq x \leq 4$, $0 \leq y \leq 3$ 범위의 점을 출력하면 직사각형의 면적은 5×4 가 되어 실제 직사각형보다 큰 면적을 출력하게 된다. 이러한 점을 고려한다면 출력해야 할 점의 범위는 $0 \leq x < 4$, $0 \leq y < 3$ 이 되어야 한다.

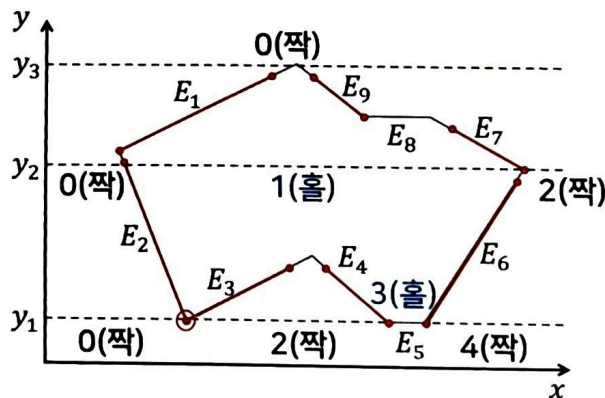


[그림 4-18] 직사각형 (0, 0)-(4, 3)

이러한 점을 고려하여 다음과 같은 방식에 따라 주사선 다각형 채우기를 적용한다.

- ① 변의 두 정점 좌표가 (x_1, y_1) 과 (x_2, y_2) 이고 $y_1 < y_2$ 인 경우 점 (x_2, y_2) 는 선분에 포함되지 않는 것으로 한다.
- ② x 축과 평행인 선분은 다각형 채우기 과정에서 제외한다.
- ③ y_i 번째 주사선의 채우기 과정에서 주사선과 다각형의 교차횟수가 홀수인 구간의 x 좌표 범위가 x_a 부터 x_b 까지라면, $\lceil x_a \rceil$ 부터 $\lceil x_b \rceil - 1$ 까지의 픽셀들을 내부 픽셀로 한다. 여기서 $\lceil n \rceil$ 은 천장함수의 값(n 보다 크거나 같은 최소 정수)이다.

[그림 4-19]는 ①과 ②에 따라 [그림 4-17]의 다각형을 구성하는 변들을 조정한 것이다. 이제 각 주사선 단위로 홀-짝 규칙을 적용하면 다각형의 내부와 외부 경계를 찾



[그림 4-19] 다각형 꼭짓점의 처리(⊙는 2개의 점임)

을 수 있으며, ③의 범위에 속하는 픽셀들을 그리면 된다.

다각형의 처리를 위해 다각형의 변을 구성하는 선분을 저장하는 선분 테이블(edge table)을 만든다. 선분 테이블에는 각 주사선 수에 해당되는 버킷이 있으며, 각 버킷에는 그 주사선에서 시작하는 선분들의 리스트가 연결된다. 단, 수평선분은 제외한다. 각 선분의 항목은 작은 y 좌표에 해당되는 주사선의 버킷에 연결되며, 한 버킷 내에서는 선분의 x 좌표 순서에 따라 선분 항목들이 정렬된다.

어떠한 선분의 양 끝점이 각각 (x_1, y_1) 과 (x_2, y_2) 이고 $y_1 < y_2$ 인 경우, 선분 항목에는 다음과 같은 내용이 저장된다.

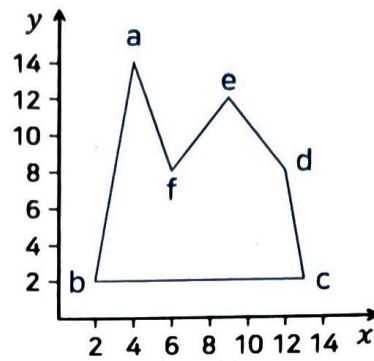
$x_{\min}$	$y_{\max}$	Δx	ptr
------------	------------	------------	-----

- $x_{\min}$: y 좌표가 작은 끝점의 x 좌표(x_1)
- $y_{\max}$: 나머지 끝점의 y 좌표(y_2)
- Δx : 다음 주사선으로 넘어갈 때 사용할 x 증분[기울기의 역수, 즉 $(x_2 - x_1)/(y_2 - y_1)$]
- ptr: 다음 선분 항목을 가리키는 포인터

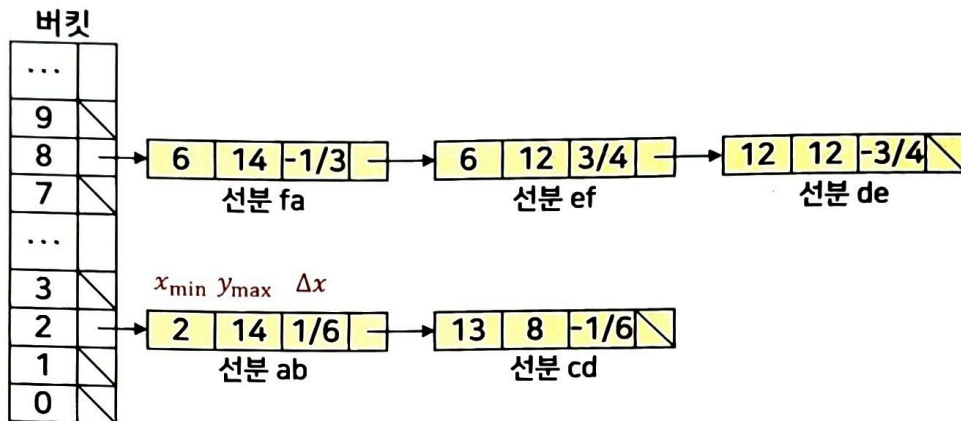
[그림 4-20]은 이와 같이 저장된 선분 테이블의 예를 보여 준다.

다각형 채우기는 주사선 단위로 이루어지는데, 이전 주사선에 대한 처리와 연속성이 존재한다. 어떠한 선분이 주사선 y_i 와 만나는 교점의 x 좌표는 이전 주사선 y_{i-1} 과 만나는 x 좌표에 Δx 를 더한 값이다.

주사선 다각형 채우기 알고리즘에서는 현재 주사선과 교차하는 선분의 목록을 관리하는데, 이를 활동 선분 목록(active edge table)이라고 한다. 각 주사선에서는 그 주사선의 버킷에 연결된 선분 항목을 활동 선분 목록에 추가하고, 그 주사선의 y 좌표가 $y_{\max}$ 에 해당되는 선분 항목을 제거한다.



(가) 처리할 다각형

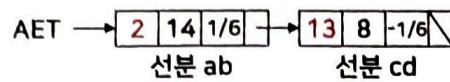


(나) 다각형의 선분 테이블

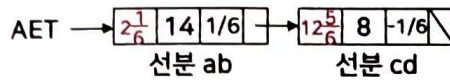
[그림 4-20] 선분 테이블

[그림 4-21]의 (가)는 2번 주사선의 활동 선분 목록으로 선분 ab와 선분 cd의 항목이 포함된다. 이 주사선에서는 $2 \leq x \leq 12$ 범위의 픽셀에 점을 그린다. [그림 4-21]의 (나)는 다음 주사선인 3번 주사선에서 선분 ab의 Δx 는 $\frac{1}{6}$ 이므로 x 좌표는 $2\frac{1}{6}$ 이 되고, 선분 cd는 $-\frac{1}{6}$ 만큼 변화한 $12\frac{5}{6}$ 가 된다. 그 결과 $3 \leq x \leq 12$ 범위의 픽셀에 점을 그린다.

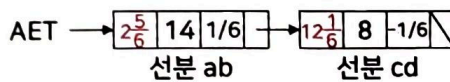
7번 주사선을 처리한 후 8번 주사선으로 넘어가면 선분 cd는 $y_{\max}$ 가 8이므로 활동 선분 목록에서 제거된다. 대신 선분 테이블의 8번 주사선 버킷에 포함된 선분 fa, 선분 ef, 선분 de가 활동 선분 목록에 추가된다. [그림 4-21]의 (다), (라)를 통해 그 과정을 볼 수 있다.



(가) 2번 주사선



(나) 3번 주사선



(다) 7번 주사선



(라) 8번 주사선

[그림 4-21] 활동 선분 목록

4.3.2. 불규칙한 경계의 영역 채우기

다각형과 같이 선분으로 둘러싸인 영역은 주사선 단위로 내부·외부를 판별하는 방법을 적용할 수 있다. 그러나 때로는 어떠한 영역이 이처럼 잘 정의된 경계에 의해 구분되지 않는 경우가 있다. 흔히 페인트 패키지에서 픽셀 그림을 그릴 때 이러한 형태의 영역 채우기를 많이 하는데, 영역 내부의 임의 지점을 선택하면 이 점부터 시작하여 경계에 이를 때까지 한 픽셀씩 내부를 ‘칠하는’ 방식으로 영역을 채운다.

이러한 채우기 방식에서 영역을 판별하는 것은 다음 두 가지 방법 중 하나이다.

첫째, 지정된 색을 만나면 영역의 경계로 판단하는 것이다. 이는 영역이 동일한 색의 경계로 둘러싸인 경우에 적용할 수 있는 방법으로, [그림 4-22]의 (가)에 이 방법을 적용한 결과는 (나)와 같다.

둘째, 지정된 색과 다른 색을 만나면 영역의 바깥으로 판단하는 것이다. 이러한 방법은 영역이 동일한 색으로 구성되어 있는 경우 사용할 수 있으며, [그림 4-22]의 (가)에 적용하면 (다)와 같은 결과를 얻는다.

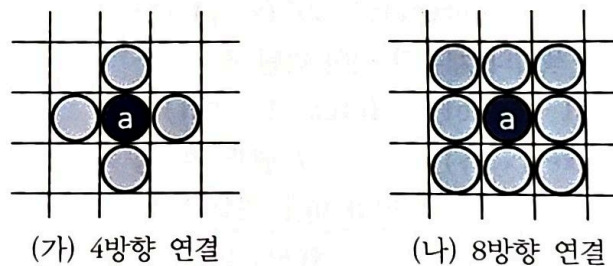


[그림 4-22] 영역의 정의 방법에 따른 영역 채우기 결과

(1) 경계선 채우기 알고리즘

경계선 채우기(boundary fill) 알고리즘은 하나의 색으로 이루어진 경계로 둘러싸인 영역을 채울 때 적용할 수 있다. 영역을 채우는 방법은 지정된 내부 영역의 픽셀로부터 시작하여 경계선의 색을 만날 때까지 인접 픽셀을 원하는 색 또는 패턴으로 채우는 방식이다.

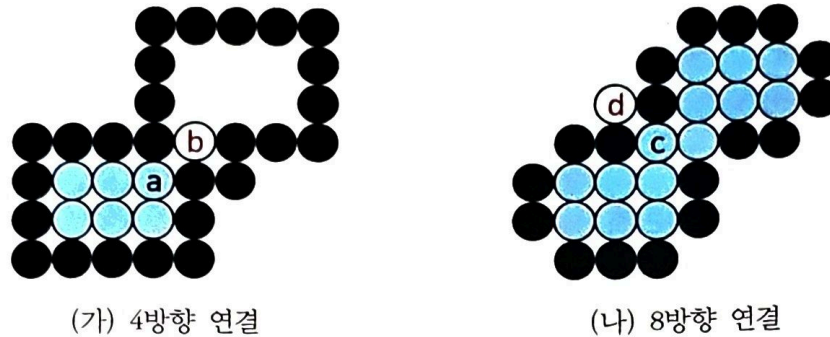
사각형 픽셀 구조를 갖는 래스터 그래픽스 방식에서 '인접'한 이웃 픽셀을 정의하는 방법으로는 4방향 연결(4-connected)과 8방향 연결(8-connected)이 있다. 4방향 연결은 [그림 4-23]의 (가)와 같이 픽셀 a에 가장 가까운 4개의 픽셀만 이웃으로 정의한 것이고, 8방향 연결은 (나)와 같이 픽셀 a를 둘러싼 8개의 픽셀을 모두 이웃으로 정의한 것이다.



[그림 4-23] 픽셀 a에 대한 이웃의 정의

만일 영역의 경계선이 4방향 연결에 따라 정의되었다면 내부 영역을 채울 때는 8방향 연결을 적용해야 한다. 그렇지 않으면 [그림 4-24]의 (가)와 같이 점 a와 b가 서로 연결되지 않아 일부 영역이 채워지지 않을 수 있다. 반면에, 경계선이 8방향 연결에 따라 정의되었다면 내부 영역을 채울 때는 4방향 연결을 적용해야 한다. 그렇지 않으면

[그림 4-24]의 (나)와 같이 영역 내부의 점 c와 외부의 점 d가 서로 연결된 점이 되어 영역 외부까지 채우게 되는 문제가 발생한다.



[그림 4-24] 영역 경계선의 연결성과 영역 내부 연결성의 관계

다음은 경계선 채우기 알고리즘을 순환적(recursive) 방식으로 정의한 것이다.

[소스코드 4-2] 경계선 채우기 알고리즘

```

1 // x, y: 픽셀 좌표, fCol: 채우기 색, bCol: 경계선 색
2 void boundaryFill4(int x, int y, int fCol, int bCol) {
3     {
4         int intCol;
5         getPixel(x, y, intCol); // (x, y) 위치의 픽셀 색을 읽음.
6         // 경계선 색이나 채우기 색이 아닌 경우
7         if (intCol != bCol && intCol != fCol) {
8             setPixel(x, y); // 현재 위치를 채우기 색으로 그림.
9             // 4방향 연결성에 따라 이웃 픽셀에 경계선 채우기를 적용
10            boundaryFill4(x+1, y, fCol, bCol);
11            boundaryFill4(x-1, y, fCol, bCol);
12            boundaryFill4(x, y+1, fCol, bCol);
13            boundaryFill4(x, y-1, fCol, bCol);
14        }
15    }

```


(2) 범람 채우기 알고리즘

범람 채우기(flood fill) 알고리즘은 동일한 색으로 구성된 영역을 채우기 위한 알고리즘이다. 영역을 채우는 방법은 지정된 내부 영역의 픽셀로부터 시작하여 이웃 픽셀이 내부 색과 같다면 지정된 색 또는 패턴으로 채우는 것을 반복하는 방식이다.

다음은 범람 채우기 알고리즘을 순환적 방식으로 정의한 것이다.

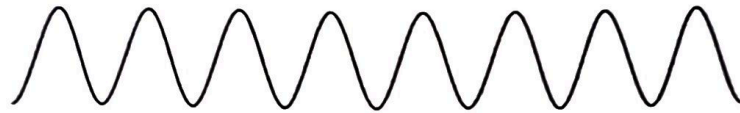
[소스코드 4-3] 범람 채우기 알고리즘

```
1 // x, y: 픽셀 좌표, fCol: 채우기 색, intCol: 내부 색
2 void floodFill4(int x, int y, int fCol, int intCol) {
3 {
4     int currCol;
5     getPixel(x, y, currCol); // (x, y) 위치의 픽셀 색을 읽음
6     // 현 위치의 색이 영역 내부 색인 경우
7     if(currCol == intCol) {
8         setPixel(x, y); // 현재 위치를 채우기 색으로 그림
9         // 4방향 연결성에 따라 이웃 픽셀에 경계선 채우기를 적용
10        floodFill4(x+1, y, fCol, intCol);
11        floodFill4(x-1, y, fCol, intCol);
12        floodFill4(x, y+1, fCol, intCol);
13        floodFill4(x, y-1, fCol, intCol);
14    }
15 }
```

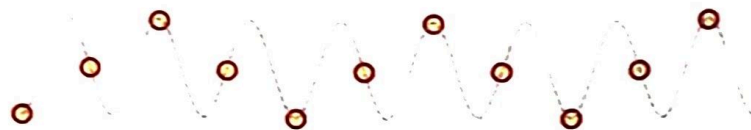
4.4 안티에일리어싱

래스터 알고리즘에 의해 생성되는 선분을 비롯한 기타 원시요소들은 표본화 과정에서 물체상의 좌표점을 불연속적인 정수 픽셀 위치로 디지털화하기 때문에 톱니 모양(또는 계단 모양)의 외형을 갖게 된다. 이와 같은 저주파 표본화(low-frequency sampling, 또는 undersampling)로 인한 정보의 왜곡을 에일리어싱(aliasing)이라고 한다.

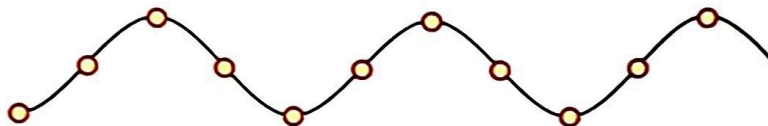
에일리어싱은 신호처리의 표본화 이론에서 유래된 단어로, [그림 4-25] (가)의 신호의 주파수보다 낮은 주파수로 표본화하면 (나)와 같이 원래 신호와는 다른 왜곡된 신호를 얻게 되는 현상을 말한다.



(가) 원본 신호



(나) 표본화된 신호



(다) 복원된 신호

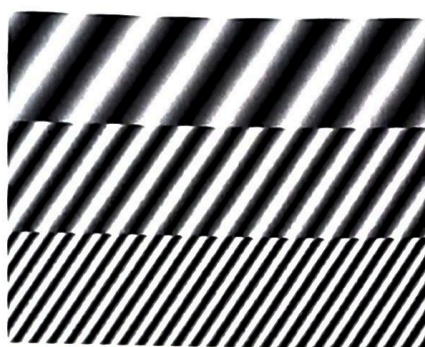
[그림 4-25] 에일리어싱 효과

표본화 이론에서는 에일리어싱에 의한 정보 손실을 피하기 위해 표본화 주파수를 나이퀴스트 표본화 주파수[Nyquist sampling frequency 또는 나이퀴스트 표본화율(Nyquist sampling rate)] f_s 이상으로 표본화한다. 나이퀴스트 표본화 주파수 f_s 는 원래 신호의 가장 높은 주파수 $f_{\max}$ 의 두 배의 주파수로 정의된다.

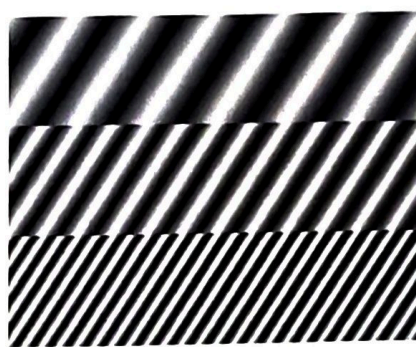
$$f_s = 2f_{\max} \quad (4-6)$$

그런데 만일 이보다 낮은 주파수로 표본화하면 에일리어싱에 의해 원래의 정보에 손실이 발생하게 된다. 래스터 그래픽스 시스템에서 영상은 이산적인 픽셀에 의해 구성되는데, 만일 표본의 수가 표현하고자 하는 그림에 비해 낮으면 에일리어싱이 발생한다. 직선이나 곡선, 물체의 경계에서는 매끄러운 그림 대신 계단형의 거친 그림이 만들어진다.

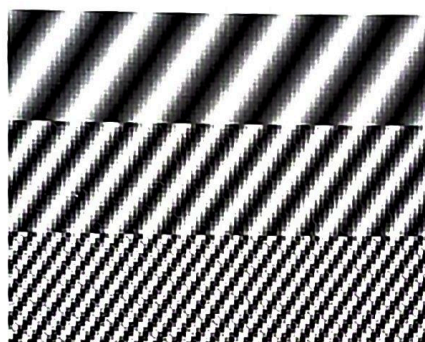
[그림 4-26]은 동일한 영상을 몇 개의 표본으로 표현했는가에 따라 보이는 모습의



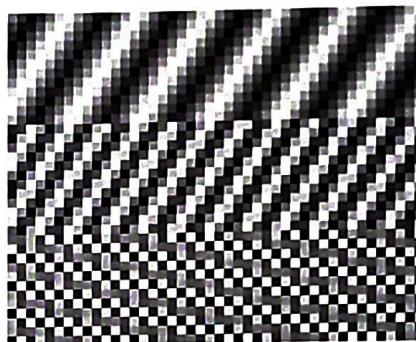
(가) 352×288



(나) 176×144



(다) 88×72



(라) 44×36

[그림 4-26] 영상의 해상도와 에일리어싱

차이를 보여 준다. [그림 4-26]의 (가)는 352×288개의 픽셀로 표현한 영상이고, (나)~(라)는 (가)와 동일한 크기의 영역에 각각 175×144, 88×72, 44×36개의 픽셀로 표현한 영상이다. 픽셀의 수가 적을수록 영상의 품질은 점점 떨어져 매끄럽던 경계가 계단형으로 거칠게 표현되고 있으며, [그림 4-26]의 (라)에서는 원래의 패턴과 다른 왜곡된 패턴이 보이기도 한다.

해상도를 높이면 에일리어싱을 어느 정도 완화할 수 있다. 그러나 많은 메모리와 고속의 그래픽스 처리 및 데이터 전송이 가능해야 한다. 최근 하드웨어의 용량 및 처리 속도가 높아지기는 했지만 여전히 에일리어싱 문제가 발생하고 있다.

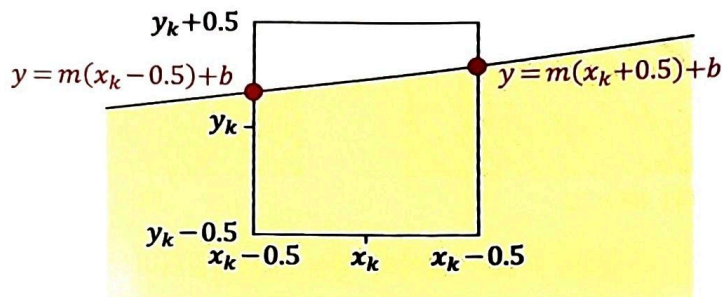
높은 해상도가 필요한 그림을 이보다 낮은 해상도로 표현할 때 해상도 차이로 인한 정보 손실은 막을 수 없지만 정보의 왜곡을 방지하여 에일리어싱을 줄이는 기법이 있는데, 이를 안티에일리어싱(anti-aliasing)이라고 한다. 컴퓨터 그래픽스에서 안티에일리어싱을 위해 사용할 수 있는 기본적인 방법으로는 영역 샘플링, 슈퍼 샘플링, 후 필터링 등이 있다.

4.4.1. 영역 샘플링

영역 샘플링(area sampling) 기법은 물체가 차지하는 픽셀 면적을 기준으로 픽셀의 음영(밝기와 색상을 포함한 값)을 조절하는 방법으로 전 필터링(prefiltering)이라고도 한다. 픽셀 중첩 영역은 물체의 경계가 개별 픽셀 경계와 어디에서 교차하는가에 따라 구한다. 예를 들어, 물체가 픽셀의 1/2을 차지하면 픽셀의 음영을 1/2로, 1/3을 차지하면 픽셀의 음영을 1/3로 조절한다.

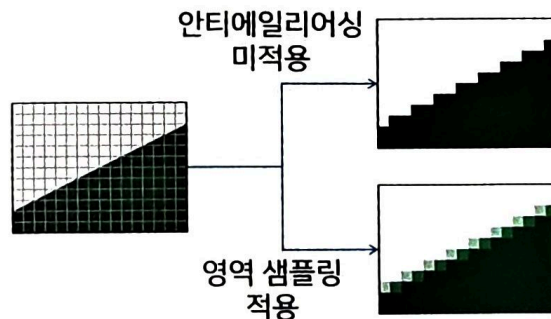
[그림 4-27]에서 물체가 차지하는 픽셀의 면적은 식 (4-7)과 같다.

$$Area = mx_k + b - y_k + 0.5 \quad (4-7)$$



[그림 4-27] 픽셀의 면적 계산

[그림 4-28]은 영역 샘플링을 적용한 결과를 보여 주는 예이다. 다각형의 경계에서 전 경색과 배경색이 다각형이 차지하는 픽셀의 면적에 따라 혼합되어 음영의 변화가 부드럽게 표현된 것을 볼 수 있다.



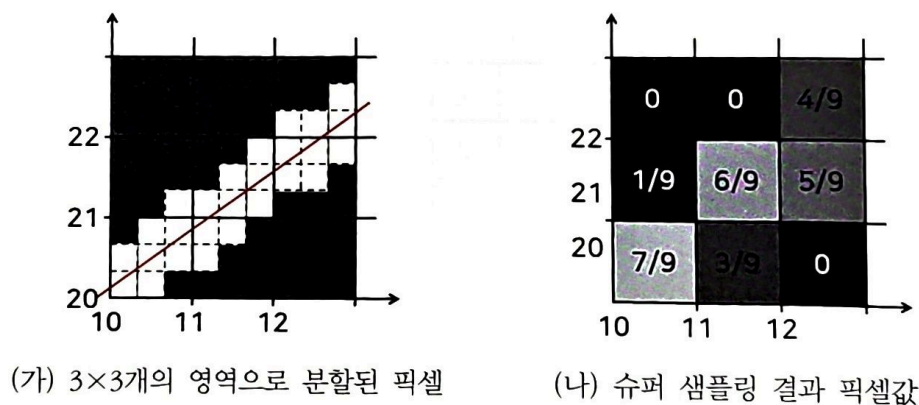
[그림 4-28] 영역 샘플링을 이용한 안티에일리어싱

영역 샘플링은 래스터 스캔으로 변환된 기하학적 모양의 물체에 적용되며, 각 픽셀 영역 내에 놓인 물체에 기반하여 각 픽셀의 평균 밝기를 계산한다. 그러나 다각형이 아닌 모양의 경우, 영역 샘플링은 계산에 많은 비용이 소요될 수 있다.

4.4.2. 슈퍼 샘플링

신호를 너무 적은 횟수로 표본화해서 발생하는 에일리어싱 현상을 감소시키기 위한 방법으로, 실제보다 더 미세한 격자로 구성된 것처럼 간주하여 표본화율을 증가시키는 것이다. 그다음 더 미세하게 나눈 이 격자의 여러 표본점을 사용하여 각 스크린 픽셀에 대한 적절한 음영을 결정할 수 있다. 이와 같이 디스플레이할 해상도보다 더 높은 해상도로 물체를 표본화한 결과를 이용하여 원래 해상도의 픽셀 음영을 구하여 디스플레이하는 것을 슈퍼 샘플링 안티에일리어싱(Super Sampling Anti-Aliasing: SSAA, 또는 Full-Scene Anti-Aliasing: FSAA)이라고 한다.

[그림 4-29]는 하나의 픽셀을 가로와 세로로 3개씩 총 9개의 서브픽셀로 샘플링하여 각 서브픽셀의 음영을 구한 후 이들의 평균으로 픽셀의 음영을 구하는 예를 보여 준다.



[그림 4-29] 슈퍼 샘플링

SSAA는 더 높은 해상도에서 많은 수의 서브픽셀 계산을 해야 하므로 훨씬 큰 비디오 메모리가 필요하고, 매우 많은 계산을 해야 하는 단점이 있다. 이러한 문제를 개선하기 위한 여러 가지 변형된 기법이 있다.

다중샘플 안티에일리어싱(MultiSample Anti-Aliasing: MSAA)은 역시 각 픽셀에 대해

다수의 서브샘플을 이용하여 픽셀의 음영을 결정하는 기법이다. 예를 들어, 어떠한 다각형 영역의 음영을 구할 때, 이 영역에 해당되는 각 픽셀에 대해 다수의 서브샘플 중 어느 것이 다각형에 포함되는지 결정한다. 그렇지만 SSAA처럼 모든 서브샘플에 대해 음영을 구하는 것이 아니라 해당 픽셀에 대해서는 한 번만 음영을 구하며, 그 값을 다각형에 속하는 서브샘플 수에 비례하여 픽셀의 음영에 반영한다. 따라서 SSAA에 비해 계산시간을 크게 절약할 수 있어 실시간 응용에서 유리하다. 그렇지만 MSAA는 다각형의 경계 부분의 안티에일리어싱에 중점을 두고 있어 다각형 내부가 텍스처와 같이 변화가 많은 패턴으로 구성되어 있을 경우 에일리어싱이 발생할 수도 있다.

4.4.3. 후 필터링

영상이 만들어진 후 안티에일리어싱 기법을 적용하여 영상을 개선하는 후 필터링(postfiltering) 방법도 있다. 후 필터링에서 픽셀값은 이웃하는 픽셀들을 포함한 가중치 평균으로 계산된다. 가중치는 필터 마스크로 구현되며, 가운데 픽셀에 가중치가 많이 주어진다. [그림 4-30]은 몇 가지 마스크의 예이다.

$\frac{1}{8}$

0	1	0
1	4	1
0	1	0

$\frac{1}{16}$

1	2	1
2	4	2
1	2	1

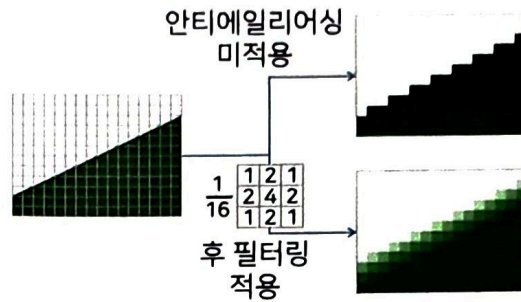
$\frac{1}{81}$

1	2	3	2	1
2	4	6	4	2
3	6	9	6	3
2	4	6	4	2
1	2	3	2	1

[그림 4-30] 여러 가지 필터 마스크

필터를 적용하는 것은 값을 구하려는 픽셀이 마스크의 중앙에 위치하도록 배치한 후 마스크의 각 가중치를 해당되는 위치의 샘플에 곱한 후 전체를 더한다. 이를 원도가 중치의 합으로 나눈 것이 처리결과 픽셀값이다.

[그림 4-31]은 후 필터링을 적용한 결과를 보여 주는 그림이다. 다각형의 경계 부분의 음영이 마스크의 가중치에 따라 인접 픽셀의 음영과 혼합되어 계단형의 경계가 부드럽게 표현되는 것을 볼 수 있다. 그러나 이 과정에서 번짐 현상이 발생하여 경계가



[그림 4-31] 후 필터링을 이용한 안티에일리어싱

선명하지 못하게 되는 부작용이 있다.

경계의 번짐 현상을 개선하기 위한 안티에일리어싱 기술도 있다. 음영의 변화가 큰 에지 영역을 찾아내고, 이 영역에서 에지의 방향에 맞게 주변 픽셀의 음영을 혼합하는 것이다. FXAA(Fast approXimate Anti-Aliasing), MLAA(MorphoLogical Anti-Aliasing) 등이 이러한 기법에 해당된다.

4.4.4. OpenGL 안티에일리어싱 함수

OpenGL에서 안티에일리어싱을 적용하는 방법 중 하나는 색 혼합 기능을 이용하는 것이다. 우선 다음과 같이 색 혼합 기능을 활성화해야 한다.

```
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
```

다음 glEnable 함수를 사용하여 안티에일리어싱을 활성화한다.

```
void glEnable(GLenum aaPrimitive);
```

- aaPrimitive: 안티에일리어싱을 적용할 그래픽스 기본 요소의 유형 (GL_POINT_SMOOTH, GL_LINE_SMOOTH, GL_POLYGON_SMOOTH)

단, GL_POLYGON_SMOOTH는 OpenGL의 구현에 따라서는 지원되지 않을 수도 있다. [소스코드 4-4]는 OpenGL에서 안티에일리어싱을 적용하여 다각형을 그리는 예이

다. 프로그램의 생략된 부분은 [소스코드 4-1]과 유사하다.

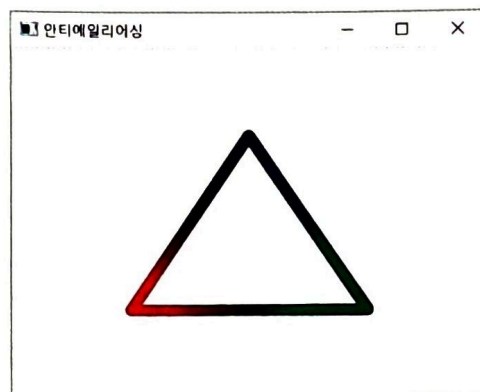
[소스코드 4-4] AntiAliasing.c

```
...   ※ [소스코드 4-1]과 동일함
101   void RenderCB()
102   {
103       glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
104       glClear(GL_COLOR_BUFFER_BIT); // 백색으로 화면 지움
105
106       glEnableVertexAttribArray(0);
107       glEnableVertexAttribArray(1);
108       glBindBuffer(GL_ARRAY_BUFFER, VBO[TRIANGLE]);
109       glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE,
                           sizeof(Vertex), 0);
110       glVertexAttribPointer(1, 4, GL_FLOAT, GL_FALSE, sizeof(Vertex),
                           (const GLvoid*)sizeof(Vec3f));
111       glEnable(GL_BLEND);
112       glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
113       glEnable(GL_LINE_SMOOTH);
114       glEnable(GL_POINT_SMOOTH);
115
116       glLineWidth(10.0f);
117       glDrawArrays(GL_LINE_LOOP, 0, 3);
118       glPointSize(10.0f);
119       glDrawArrays(GL_POINTS, 0, 3);
120
121       glDisableVertexAttribArray(0);
122       glDisableVertexAttribArray(1);
123       glFinish();
124   }
...   ※ 이후 [소스코드 4-1]과 동일함
```

111~112행에서 색 혼합을 활성화하고, 113~114행에서 선과 점 안티에일리어싱을 활성화하고 있다. 116~117행은 다각형의 변을 폭이 10인 선으로 그린다. 또한 118~119행

에서 다각형 꼭짓점에 크기가 10인 점을 그려 변의 연결 부분이 원형 연결이 되도록 하는 것을 볼 수 있다.

[그림 4-32]는 이 프로그램을 실행시킨 결과이다. 다각형의 변을 구성하는 선들의 폭이 기울기에 무관하게 동일한 것을 볼 수 있으며, 꼭짓점의 연결이 원형으로 마무리된 것을 볼 수 있다.



[그림 4-32] OpenGL의 안티에일리어싱 적용

요약

- 색의 3요소는 빛의 색을 결정하는 색조(hue), 색이 순수한 스펙트럼색에 얼마나 가까운가를 나타내는 채도(saturation), 빛의 전체적인 에너지 크기에 해당되는 명도(brightness)이다.
- 삼원자극이론: 서로 다른 파장의 빛에 가장 높은 흡수율을 보이는 세 종류의 원추세포에 흡수되는 빛의 양에 의해 색을 느끼며, 서로 다른 스펙트럼의 색이라도 이 흡수량이 같으면 동일한 색으로 인지한다.
- R, G, B 삼원색을 혼합하여 표현할 수 있는 색역은 사람의 가시광선의 색의 일부만 포함한다.
- CIE XYZ 색 모델은 인간의 시각 특성에 맞추어 정의된 가상의 표준원색을 이용하여 평균적인 인간의 눈에 보이는 모든 색을 표현할 수 있다.
- RGB 색 모델은 빨강, 초록, 파랑을 삼원색으로 하는 가산색 모델이고, CMY 색 모델은 청록, 자홍, 노랑을 삼원색으로 하는 감산색 모델이다. HSV 색 모델은 색조, 채도, 명도로 색을 표현한다.
- 많은 그래픽스 패키지는 현재 속성값의 목록을 보관하고, 이를 갱신할 수 있는 함수를 제공한다. 그래픽스 기본 요소를 그릴 때는 이 속성 목록의 현재 값을 적용한다.
- 비트맵 패턴의 채우기는 정해진 크기의 마스크를 반복적으로 이어 붙이는 타일 붙이기 형식으로 처리된다.
- 타일 붙이기는 원도의 원점이나 도형의 경계상자 모서리점을 기준으로 적용한다.
- 주사선 다각형 채우기
 - ① 각 주사선 단위로 홀-짝 규칙을 적용한다.
 - ② 주사선이 꼭짓점을 만나는 경우의 처리를 위해 각 변에서 y 축 좌표의 값이 큰 꼭짓점 좌표는 제외한다. 또한 수평 선분은 처리에서 제외한다.
 - ③ 다각형 선분 테이블에는 각 주사선 단위로 그 주사선 y 좌표에서 시작되는 변들의 리스트를 x 좌표 순서에 따라 저장한다.

- ④ 활동 선분 목록은 현재 처리 중인 주사선과 교차하는 변들의 리스트를 저장한다.
- 경계선 채우기 알고리즘은 정해진 색으로 되어 있는 경계로 둘러싸인 영역 내부를 지정된 색으로 채운다.
- 범람 채우기 알고리즘은 동일한 색으로 되어 있는 영역을 지정된 색으로 채운다.
- 에일리어싱은 낮은 해상도로 인한 정보 왜곡이다. 래스터 그래픽스 시스템에서 선분이나 영역 경계가 계단형으로 출력되는 현상이 나타난다.
- 안티에일리어싱은 에일리어싱 현상을 완화하기 위한 기술로, 영역 샘플링, 슈퍼 샘플링, 후 필터링 등이 있다.
- 영역 샘플링은 물체가 차지하는 픽셀 면적에 따라 픽셀의 값을 결정한다.
- 슈퍼 샘플링(SSAA)은 하나의 픽셀을 여러 개의 작은 픽셀로 분할하여 물체 영역이 차지하는 서브픽셀의 수에 따라 픽셀의 색을 결정한다. 다중샘플 안티에일리어싱(MSAA)은 SSAA에 비해 계산시간 및 메모리 소비를 줄일 수 있는 기법으로, 다각형의 경계 부분에 중점을 두어 안티에일리어싱을 수행한다.
- 후 필터링은 이웃 샘플의 가중치 평균으로 픽셀값을 결정한다. FXAA나 MLAA는 에지의 방향에 맞게 필터링하는 방법으로 번짐 현상을 개선한다.
- OpenGL의 그래픽스 기본 요소 속성 관련 함수

함수	내용
<code>glEnable(GL_BLEND);</code> <code>glDisable(GL_BLEND);</code>	색 혼합의 활성화 및 해제
<code>void glBlendFunc(GLenum sFactor, GLenum dFactor);</code>	소스 및 목적지 색 혼합계수 설정
<code>void glPointSize(GLfloat size);</code>	점 크기 설정
<code>glEnable(GL_POINT_SMOOTH);</code> <code>glDisable(GL_POINT_SMOOTH);</code>	점의 안티에일리어싱 활성화 및 해제
<code>void glLineWidth(GLfloat width);</code>	선 폭 속성 설정
<code>glEnable(GL_LINE_SMOOTH);</code> <code>glDisable(GL_LINE_SMOOTH);</code>	선의 안티에일리어싱 활성화 및 해제